Documentation for the task

Goal and general workflow

The goal of this project is to build an pipeline that processes and moves raw data coming from an external ATS system (source). The pipeline covers the path from raw data ingestion through validation, staging, and - finally - aggregation into a data mart for downstream use.

In order to achieve this, the workflow focused on a few aspects, specifically outlined in the task description. These areas are presented with more detail in this document:

- **Data modeling** - designing source and warehouse schemas with appropriate tables, constraints and relationships among these tables.
- **Data processing** - moving data incrementally from the source (ATS) layer through staging area and, finally, into a data mart, while handling duplicates and data quality issues.
- **Data quality (logging)** - capturing DQ-related issues detected at each pipeline run, therefore improving monitoring/observability in general.
- **Analytical queries** - computing hiring metrics (as outlined in *Part B* of the task description).

To simulate a „realistic“ scenario, mock data was generated and inserted into the source tables. Moreover, some of inserted records intentionally contain data quality issues and the pipeline is designed to detect, log, and exclude them from the downstream layers. The pipeline is also designed with idea to support idempotent, incremental processing, meaning it should be able to run multiple times without issues (e.g. duplicate results), and only processes or (re)evaluates records that are new/changed in the meantime.

Tech stack

The task is carried out by using PostgreSQL (schemas, data processing and query logic are pure SQL), in combination with Docker.

Please note that Docker is used just to simplify environment setup and it is not required - *the docker-compose.yaml* file just launches up a single PostgreSQL container (*depd-postgres-storage*), exposed on port 5433, with the DB (named *ats_data*).

The outcome

The SQL code (scripts) is provided in the GitHub repository. In addition to this, the documentation presented here aims to cover rationale behind the utilized approach and to provide some context around decisions made during the implementation of the project

solution. In the following text, main aspects of the workflow will be described more precisely.

Data modeling

From the standpoint of (physical) data modeling, two schemas have been introduced with aim to separate concerns across the pipeline:

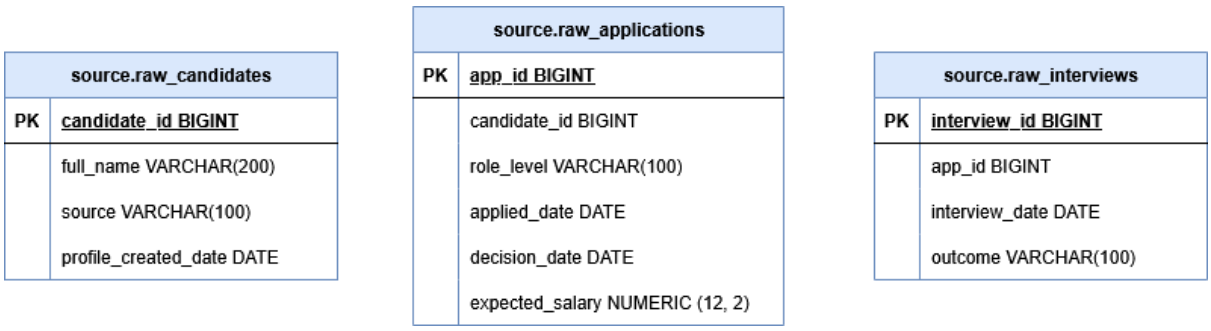
- *source* schema - for storing raw data, as received from source/ATS
- *dwh* schema - represents structured data warehouse schema, contains staging tables, helper/log tables and the data mart

Details about implementation of these schemas and entities are provided below. The code is provided in *sql\schema_init.sql* file in GitHub repo.

Source schema

The raw data coming from ATS/source system is stored in *source* schema, where tables are defined according to the task description. Data is kept in „as-is“ format - meaning no business rules or constraints have been applied at this stage of implementation. The idea is to simulate the „realistic“ scenario where data is stored exactly as it arrives from the external system, even if these records are flawed, rather than rejecting them.

Please note that the logical 1:M relationships exist across the tables (one candidate may have many applications; one application may have many interviews), but no FK constraints are enforced at this layer, keeping data in as-is format.



Picture 1: ERD diagram of the tables belonging to source schema

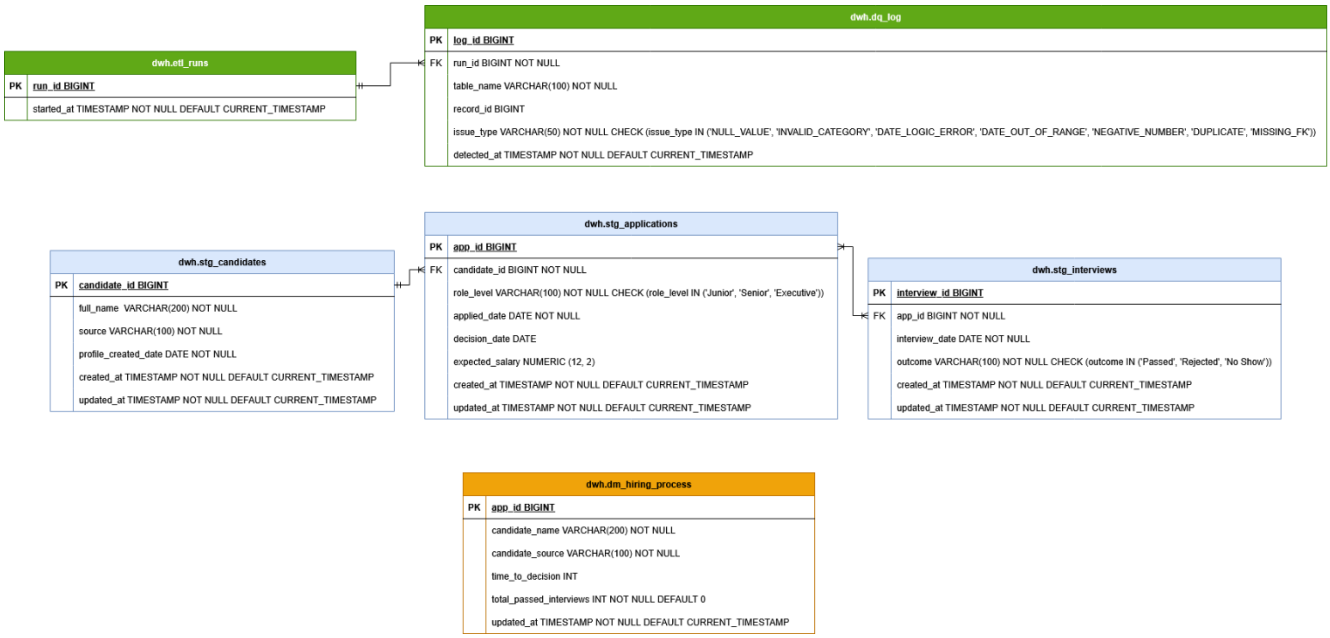
DWH schema

The DWH schema actually represents the structured warehouse layer for data. Logically, it might be considered as 3 groups:

1. **Staging tables** - *stg_candidates*, *stg_applications* and *stg_insterviews* - structurally mirrors the source tables; but they enforce business rules (through e.g. NOT NULL, CHECK constraints, date logic etc.). Only records that pass these validations are loaded in staging tables. These tables persist across pipeline

executions and serve as the validated, clean layer of the DWH. Metadata/technical columns are also added (*created_at*, *updated_at*).

2. **„Helper“ tables** - *etl_runs* and *dq_log* - are designed to support observability across the pipeline. Namely, *etl_runs* captures a timestamp for each execution, while *dq_log* catches all detected DQ issues (if any), linked back to the pipeline run.
3. **Data mart** - *dm_hiring_process* - represents the final, aggregated output, consisting of one row per application, covering the required metrics.



Picture 2: ERD diagram of the tables belonging to dwh schema. Note: green – helper tables for DQ logs and pipeline runs; blue – staging tables, orange – DM table

Data flow and data processing

Pipeline overview

From the technical standpoint, SQL code process data through the pipeline in the few consecutive steps. The code is provided in *sql\etl_source_to_datamart.sql* file placed in GitHub repo.

First, it captures and registers the pipeline run by inserting a new record into *etl_runs* table and capturing the generated *run_id* in a temp table (named *current_run*). Since *etl_runs* table is tied to *dq_log* table they represent the baseline for observability of pipeline executions and data quality issues found during that.

Then, it examines data in source tables against a set of DQ checks and appends any detected issues to the *dq_log* table. Each unique DQ issue is logged exactly once, on the

run where it was first detected. Detailed description is given in the section ***DQ log design and execution tracking*** below.

Pipeline then upserts clean and valid records into staging tables - *stg_candidates*, *stg_applications* and *stg_interviews*.

For *stg_candidates*, when multiple *raw_candidates* rows share the same composite/natural key (*full_name*, *source*, *profile_created_date*), only the earliest record is loaded. Any records with NULL values in *full_name*, *source* or *profile_created_date*, or with *profile_created_date* outside the expected range (2020-2030), are excluded.

For *stg_applications*, the same deduplication logic applies, with the composite key consisting of *candidate_id*, *role_level* and *applied_date*. Records are additionally excluded if: *candidate_id*, *role_level* or *applied_date* are NULL; *role_level* is not in the allowed list of predefined categories; *decision_date* comes before *applied_date*; any date falls outside the expected range; *expected_salary* is negative. The upsert joins directly to *stg_candidates* which should ensure that only applications whose candidate already passed DQ checks and landed in staging are loaded. It should keep the layer boundaries clean and enforce referential integrity against the already-validated data.

For *stg_interviews*, deduplication is based on the composite key of *app_id*, *interview_date* and *outcome*. Excluded records are those with NULL values in *app_id*, *interview_date* or *outcome*; *outcome* not in the allowed list of categories; *interview_date* before the linked application's *applied_date*; or *interview_date* out of the expected range. The upsert joins to *stg_applications* to enforce that only interviews with a valid application are loaded.

Deduplication strategy

Duplicate detection applies to all source tables. A duplicate is defined as two (or more) rows sharing the same composite/natural key, defined per each table. Deduplication logic relies on window functions (ROW_NUMBER) and retains only rows with lowest PK (*_id*) value. Other rows are discarded and treated as duplicates before staging INSERT clause executes.

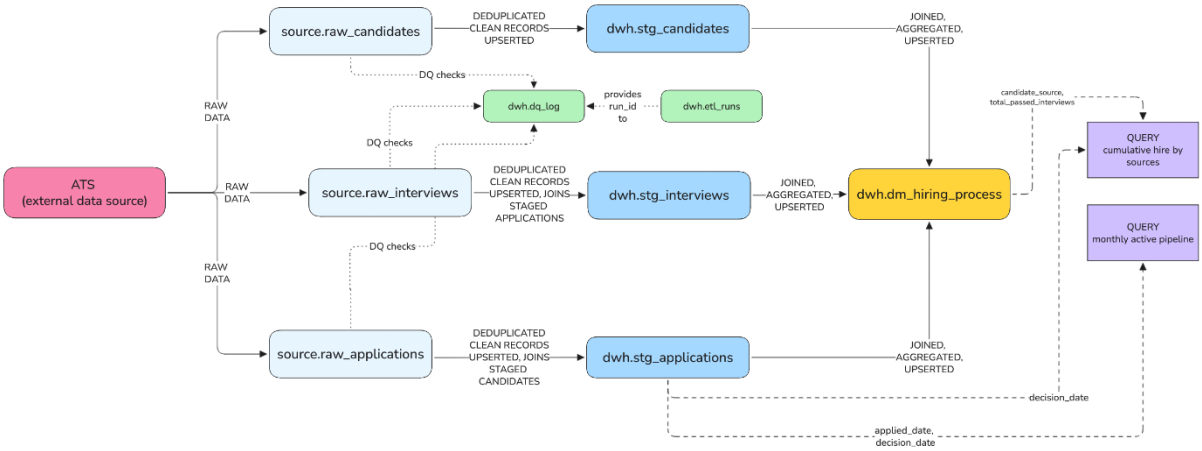
The reason for this approach is that lowest *_id* actually represents earliest inserted record - original submission - meaning the record that arrived first from external source system. In other words, higher value of *_id* could be the re-submission, an accidental double entry, the system retry etc. Moreover, since *_id* values are sequential (due to the GENERATED ALWAYS AS IDENTITY clause) this becomes a deterministic, and re-submissions are treated as a noise.

Data mart

Finally, the pipeline upserts aggregated metrics into data mart, *dm_hiring_process*. As per task description, *dm_hiring_process* represents the final output of the pipeline,

consisting of one row per application. All metrics are recalculated on every run. Thus, the data mart always reflects the current state of simultaneous hiring processes, which means - for example - active applications that receive a *decision_date* between runs, or applications that accumulate new interviews, are updated on the next execution.

Similarly as in case of staging tables, metadata/technical column *updated_at* is added with aim to provide insight into data freshness/staleness. Please note that data mart is made as physical table rather than a view or materialized view.



Picture 3: Graphical overview of the pipeline

Also, please note that BEGIN;/COMMIT; blocks have been used as an example of good practice to isolate potential failures and preserve partial progress across the pipeline steps. DQ checks are grouped into one transaction per source table, while each staging upsert and the data mart upsert run in their own individual transactions.

Incremental loading and upsert strategy

The pipeline has been made with idea to ensure idempotency, so it can be safely executed whether to pick up new records coming from source/ATS and/or to reflect updates to existing ones.

This is achieved by utilizing upsert logic (ON CONFLICT DO UPDATE...), so if the record already exists, some fields are overwritten with the latest values (and the *updated_at* columns are being refreshed, too). Otherwise - if the record is new - it is inserted normally.

The upsert follows an overwrite-on-update pattern, conceptually similar to SCD type 1 from classical dimensional modeling paradigm - no history is retained and old values are simply overwritten with the latest ones.

Another option considered during implementation was - conceptually - SCD2-like logic, which would close/mark existing record(s) as inactive and insert new row(s). However, implementation of such logic was not justified by the requirements of this task. Also, it would include more complexity (e.g. handling of keys, additional columns like *valid_to*/*valid_from*, as well as *is_active*/*is_current* flags). If historical tracking of e.g. candidate profile changes were to become a requirement, migrating to this type of logic would be the natural next step, especially if the classic dimensional modeling is utilized.

Accordingly, the following per-table reasoning serves as the baseline supporting the implementation of described approach:

1. In the context of the standard hiring process - and the task description - the table with candidates data (*stg_candidates*) is unlikely to change meaninfully over time (e.g. candidate full name or source columns). Even if update arrives at source, overwritting is probably correct approach - since the talent acquisition/HR team is probably more interested in most current and accurate version, rather than a some kind of changelog.
2. For the applications data (*stg_applications*) the most important incremental update scenario should be the one when active application (meaning no *decision_date*) receives a final decision (e.g. from hiring manager). Therefore, upsert simply fills the previously NULL value.
3. Given the task description – and from a logical perspective - interview records seems to be generally immutable once recorded. Still, the upsert allows for updates/corrections (e.g. *outcome* field) if needed. From the practical standpoint, this is demonstrated by the additional, incremental mock data sample. More precisely, when *mock_data_incremental_insert.sql* script is executed, it adds *decision_date* to some of already existing (active) applications. The next pipeline run will pick up these changes and propagate them through staging into the data mart (*dm_hiring_process*). Therefore, for some records, NULL values in *time_to_decision* are being updated to their calculated values, and *total_passed_interviews* is recalculated with any newly added interviews. Execution of this script is just for demonstration purposes, to showcase the described workflow.

Mock data and testing

Mock data samples were generated across two scripts (*sql\mock_data_insert.sql*, *sql\mock_data_incremental_insert.sql*) to test both the initial load and the incremental loading. Some of records within each batch intentionally contain data quality issues, designed to trigger every DQ check path. The second batch has some overlap with initial mock data, and it is intentionally structured to trigger every upsert path: new inserts, updates to existing rows, and recalculation of data mart metrics.

Data quality approach

The idea behind the DQ approach utilized in the task is to catch and capture data quality issues, before any data is moved to staging. From the logical standpoint, and in line with the task description, the DQ issues present in the mock data were intentionally predefined to reflect the most common issues that could come with raw data from external source system.

DQ checks

There are 7 types of data quality checks implemented:

- NULL values (NULL_VALUE) - a required field contains NULL; e.g. *full_name* in *raw_candidates* table or *applied_date* in the *raw_applications* table.
- Invalid category (INVALID_CATEGORY) – a field contains a value out of expected/allowed list defined in the task description; e.g. „Manager“ as *role_level*, or „Pending“ as interview *outcome*.
- Logical date issues (DATE_LOGIC_ERROR) – a nonsense date relationship that violates business logic, e.g. *decision_date* before *applied_date*, *interview_date* before the linked application's *applied_date*, or *profile_created_date* after the linked application's *applied_date*. Please note, the latter check is logged against the application record (not the candidate), since the mismatch is observable per individual application.
- Duplicated values (DUPLICATE) – appearance of duplicates, values appearing more than once sharing the same composite/natural key within a source table; applies to all source tables.
- Missing FKs (MISSING_FK) – a FK references to a non-existent record e.g. an application referencing a *candidate_id* that does not exist in *raw_candidates*, or an interview referencing an *app_id* that does not exist in *raw_applications*. Since no FK constraints are enforced at the source layer, the pipeline should detected orphaned records through expliciti DQ checks.
- Date out of range (DATE_OUT_OF_RANGE) – unreasonable dates failing out of the expected, predefined window (in this case, 2020-2030 window is set as reasonable baseline), e.g. dates placed in1900. or 2626. year.
- Negative numbers (NEGATIVE_NUMBER) – a numeric field contains a value that can’t logically be negative, e.g. *expected_salary* recorded as a negative number.

The full coverage of DQ checks by source table is as follows:

- **source.raw_candidates**: NULL_VALUE, DATE_OUT_OF_RANGE, DUPLICATE

- **source.raw_applications**: NULL_VALUE, INVALID_CATEGORY, DATE_LOGIC_ERROR, DATE_OUT_OF_RANGE, NEGATIVE_NUMBER, DUPLICATE, MISSING_FK
- **source.raw_interviews**: NULL_VALUE, INVALID_CATEGORY, DATE_LOGIC_ERROR, DATE_OUT_OF_RANGE, DUPLICATE, MISSING_FK

Please note that the pipeline does not raise exceptions or stop data processing when issues are detected. It simply logs the flawed records and skips them during the staging upsert step.

DQ log design and execution tracking

In line with the previously described approach, the *dq_log* table serves a kind of „helper" table for capturing and further potential investigation of detected data quality issues. Over time, it should accumulate information about the frequency and type of DQ issues - so it might be useful for detection and prevention of reccuring problems.

The design should ensure that each DQ issue is logged at most once. This is enforced both by a UNIQUE constraint on the table itself and by DQ INSERT statements that use ON CONFLICT ... DO NOTHING - meaning If an issue has already been recorded in a previous run, the insert is silently skipped. The *dq_log* table works in combination with *etl_runs* table, enabling logical audit of pipeline's behavior. The *run_id* column records the pipeline execution in which the issue was detected for first time, while the *detected_at* column captures the corresponding timestamp.

The mechanism begins by inserting a new record into *etl_runs* and capturing the generated *run_id* into a temp table (named *current_run*; by definition it is session-scoped) using a CTE with an INSERT ... RETURNING clause. This should guarantees that the *run_id* used through the DQ logging blocks always corresponds to the current session's run. The temp table is dropped and rebuilt as fresh/clean at the start of each execution, even if the pipeline is re-executed within the same session.

Ad-hoc summary queries are also provided to show quick record counts across all pipeline layers and DQ issue counts by table and issue type for the current run. Please note, these queries are placed at the very end of the main pipeline script (*etl_source_to_datamart.sql*) and commented.

Analytical queries

The code for queries is provided in *sql\queries_part_b.sql*.

1st query

The requirement from task description (part B) is to calculate the monthly active pipeline - the number of active applications for each month

The idea behind the query presented in the code is to serve as a kind of hiring pipeline monitor. Also, an application is considered as active within a specific month if that month falls between application boundaries (*applied_date*, *decision_date*; inclusive). If there is no *decision_date* yet, the assumption is that hiring process is still ongoing for this particular application, so the „upper bound“ equals to the current month. So the query considers the current partial months, too. *Generate_series* clause has been used to expand application into row per month it spans, then query counts the rows per month

2nd query

For next query, the requirement from task description (part B) is to calculate cumulative hires by source – where *decision_date* exists and the *outcome* is positive/success, per each source

The starting premise is that „hire“ is defined as an application with a non-NULL decision date and at least one *Passed* interview *outcome*. The latter criteria may seem debatable, but there is no strict requirement of explicit hired/rejected flag mentioned. Also, there is no threshold set on how many passed interviews are needed to consider outcome as successful - then at least one *Passed* value serves as a meaningful baseline - especially given that number of interview rounds may vary across roles/positions/departments/seniority levels etc.

Moreover, it is worth to mention that if there is (edge) case with „mixed outcome“ – where some interviews are marked as *Passed* and some as *Rejected* in regard to the same application - it would still be considered as „hire“ under described definition of „hire“, at least conceptually. Still, this is debatable approach.

The query covers all years present in the inserted mock data, with the cumulative hire counter resetting to zero at the start of each calendar year per source channel; no year boundary is hardcoded. The query relies on two CTEs - the first one (*hires*) filters qualifying records, extracts the hire month, and derives the hire year; the second (*monthly_hires*) counts hires per source per month per year. The final SELECT clause applies a SUM window function partitioned by both source channel and hire year, ordered by hire month, producing a running cumulative sum that grows month-by-month within each year and resets at the start of the next.

Possible different approaches, future improvements

Usage of PL/pgSQL stored procedures may make the pipeline more structured, and would allow the use of RAISE NOTICE or similar features. Also, the entire pipeline logic could be wrapped in chain of stored procedures. However, the task description makes no mention of such an approach, and the plain SQL implementation seemed like best choice for the task.

Also, it is worth nothing that current state of the code encompasses pipeline code and code for DQ checks together (file *etl_source_to_datamart.sql*). That is just for clarity and keeping the logic at the same place. From a practical standpoint, they could be split into two scripts and executed consecutively.

Additionaly, when it comes to DQ checks, the *dq_log* table could be imroved by adding additional column(s) like e.g. *details/description* or similar, where the specific column that triggered the issue could be captured, which should enable faster root-cause investigation.